

■ Python 90 Days Challenge

String Operations — Source Pages 91–98

Source: CodeWithNishchal Python Programming notes. This PDF keeps the content from pages 91–98 and removes unnecessary empty space.

6. String Operations

A string is a sequence of characters enclosed in quotes. Strings are one of the most used data types in Python because text handling is needed in almost every program.

6.1 String Creation

Strings can be created using single quotes, double quotes, or triple quotes.

```
s1 = "Hello"
s2 = 'Hello'
s3 = """Hello Python"""
```

Single and double quotes are commonly used for one-line strings. Triple quotes are used for multi-line strings.

```
message = """Python is simple.
Python is powerful.
Python is widely used."""
```

6.2 String Indexing

Indexing means accessing one character from a string. Python starts counting from 0.

```
word = "Python"
print(word[0])
print(word[3])
```

Output: P, h

For negative indexing, counting starts from the end.

```
print(word[-1])
print(word[-2])
```

Output: n, o

6.3 String Slicing

Slicing means taking a part of a string. The start index is included and the end index is excluded.

```
Syntax: string[start:end]
word = "Python"
print(word[0:3])
print(word[2:])
print(word[:4])
```

Output: Pyt / thon / Pyth

You can also use step values.

```
print(word[0:6:2])
Output: Pto
```

To reverse a string:

```
print(word[::-1])
Output: nohtyP
```

6.4 String Methods

String methods are built-in functions that work on strings. They are used for cleaning, checking, changing case, finding text, and replacing text.

lower()

"Python".lower() → "python"

upper()

"Python".upper() → "PYTHON"

strip()

" Python ".strip() → "Python"

replace()

"I like Java".replace("Java", "Python") → "I like Python"

split()

"apple,banana,mango".split(",") → ['apple', 'banana', 'mango']

find()

`"Python".find("th")` → 2. If not found, it returns -1.

count()

```
"banana".count("a") → 3
```

startswith() / endswith()

```
"Python".startswith("Py") → True
```

```
"Python".endswith("on") → True
```

6.5 String Formatting

String formatting means placing values inside a string in a clean way. Python has f-strings, `format()`, and old `%` formatting.

6.5.1 f-strings

```
name = "Aman"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

6.5.2 format()

```
print("My name is {} and I am {} years old.".format(name, age))
```

6.5.3 Old % Formatting

```
print("My name is %s and I am %d years old." % (name, age))
%s = string, %d = integer
```

6.6 String Concatenation

Concatenation means joining strings together. The + operator is used.

```
first = "Python"
second = "Programming"
print(first + " " + second)
```

Output: Python Programming

6.7 Escape Characters

\n = new line \t = tab \\ = backslash \' = single quote \" = double quote

Common Escape Characters

- ↳ In New line → `print("Hello\nPython")`
- ↳ Tab → `print("Name\tAge")`
- ↳ Backslash → `print("C:\\Users\\Admin")`
- ↳ Single quote → `print('It's Python')`
- ↳ Double quote → `print("He said \"Hello\")`

\b Backspace, \r Carriage Return, \f Form Feed, \v Vertical Tab, \a Alert/Bell

6.8 Raw Strings

Raw strings treat backslashes as normal characters. Write r before the string.

```
path = r"C:\new_folder\test"
print(path)
Output: C:\new_folder\test
```

6.9 String Multiplication

A string can be repeated using the * operator.

```
text = "Hi"  
print(text * 3)  
Output: HiHiHi
```

Common String Methods

Method	Description	Example → Result
lower()	Converts all characters to lowercase	"Python".lower() → "python"
upper()	Converts all characters to uppercase	"Python".upper() → "PYTHON"
title()	Capitalizes first letter of every word	"python programming".title() → "Python Programming"
capitalize()	Capitalizes only the first letter	"python".capitalize() → "Python"
swapcase()	Swaps uppercase and lowercase	"PyThOn".swapcase() → "pYtHoN"
strip()	Removes spaces from both ends	" Python ".strip() → "Python"
lstrip()	Removes spaces from the left	" Python".lstrip() → "Python"

Method	Description	Example → Result
<code>rstrip()</code>	Removes spaces from the right	<code>"Python ".rstrip()</code> → <code>"Python"</code>
<code>replace()</code>	Replaces one substring with another	<code>"I like Java".replace("Java","Python")</code> → <code>"I like Python"</code>
<code>find()</code>	Returns first index of substring	<code>"Python".find("th")</code> → <code>2</code>
<code>index()</code>	Returns index; raises error if not found	<code>"Python".index("th")</code> → <code>2</code>
<code>count()</code>	Counts substring occurrences	<code>"banana".count("a")</code> → <code>3</code>
<code>startswith()</code>	Checks starting substring	<code>"Python".startswith("Py")</code> → <code>True</code>
<code>endswith()</code>	Checks ending substring	<code>"Python".endswith("on")</code> → <code>True</code>
<code>split()</code>	Splits string into a list	<code>"a,b,c".split(",")</code> → <code>["a","b","c"]</code>
<code>join()</code>	Joins iterable elements into a string	<code>"-".join(["A","B","C"])</code> → <code>"A-B-C"</code>
<code>isalpha()</code>	True if all characters are alphabets	<code>"Python".isalpha()</code> → <code>True</code>
<code>isdigit()</code>	True if all characters are digits	<code>"12345".isdigit()</code> → <code>True</code>
<code>isalnum()</code>	True if letters or digits	<code>"Python3".isalnum()</code> → <code>True</code>
<code>isspace()</code>	True if all characters are whitespace	<code>" ".isspace()</code> → <code>True</code>
<code>center(width)</code>	Centers string within width	<code>"Python".center(12)</code> → <code>" Python "</code>
<code>zfill(width)</code>	Pads with leading zeros	<code>"25".zfill(5)</code> → <code>"00025"</code>